

Document 11 — Multi-Agent Systems → Project 4

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[10_agent_workflows](#) (Project 3 done)

How to Read & Practice This Document

- **What:** splitting one task across several specialized agents.
- **Why:** this is the real goal of the whole curriculum — and also the easiest place in the whole stack to overdo it for no good reason.
- **When:** when one agent's tools or context become too much to handle well, or a task truly needs specialist thinking — never just by default, because multi-agent sounds impressive.
- **How to practice:** 1. Read the pattern table below before writing any code — you should be able to name all 7 patterns from memory before starting. 2. Do the **Basic** paper-design exercise closed-book — guess the cost/speed/reliability before building anything. 3. Do **Intermediate/Real-world** by building two versions of the *same* task, and comparing the real numbers to your guess. 4. Try **Project 4** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, defend out loud — to an imagined tough senior engineer — why this task needed 4 agents and not 1. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Project 3 (Doc10) built one agent that could search, reason, pause for approval, and finish. That agent is good at one kind of job. But real work often has several genuinely different jobs stacked on top of each other — go find out what's true, figure out what it means, write it up clearly, and then check the writing is actually good — and asking one agent's single system prompt to be excellent at all four of those at once is asking a lot. That's the problem this document is really about.

The tempting mistake is to assume "more agents" is automatically better, because it sounds more advanced. It usually isn't — a single well-organized agent is cheaper, faster, and has fewer places to

break, and stays your default choice. Multi-agent design only earns its cost when a task truly has separate phases that would fight each other in one prompt, when one agent's tools or history would get overloaded, or when running independent pieces at the same time genuinely saves real time. This document walks through the seven shapes multi-agent systems come in — sequential, supervisor, hierarchical, peer-to-peer, parallel, and generator → critic — so that you're choosing a shape on purpose, with a real reason, not by default.

That's the whole story: **the patterns** are just different ways of arranging who talks to whom and in what order. The **Command tool** is simply today's clean way for one agent to say "here's my update, and here's who goes next" in a single step, replacing older, more fragile text-code routing. And the **Build Task**, Project 4, asks you to put a real version of this together: a Supervisor that routes a task to a Research agent, then an Analysis agent, then a Writer, then a Reviewer who can send the work back for changes — a real, working, multi-agent pipeline, not just a diagram of one.

Core Concepts (read this first — everything you need is here)

Topics on this page: [The real question: one agent, or many?](#) · [Every pattern below: what/why/when/trade-off](#) · [The Command tool](#) · [Shared state design](#) · [Error propagation between agents](#) · [Cost and latency](#) · [Agent-to-agent communication patterns](#)

The real question: one agent, or many?

Before any pattern below, ask the question that actually matters: **does this task genuinely need multiple agents, or would one agent with more tools do the job for less cost, less delay, and fewer ways to break?** A single agent with a large, well-organized set of tools can handle a surprising amount. Splitting into multiple agents earns its cost specifically when: the task has genuinely separate phases that would conflict if forced into one system prompt, when one agent's context would get overloaded with too many tools or too much history to reason about well, or when running independent parts at the same time gives a real speed benefit. **Why this matters more than memorizing patterns:** the failure this whole document guards against is reaching for multiple agents because it sounds advanced, then paying 3-4 times the cost and delay of a single agent, for a task that never needed splitting up.

Every pattern below: what / why / when / trade-off

- **Single agent** — one loop, all tools. *Why:* simplest to build, fix, and reason about. *When:* your default choice — use it until you have a real reason not to. *Trade-off:* cheapest and fastest, but a long tool list can make tool choices less reliable.
- **Sequential** — a fixed order, agent → agent → agent, each one's output feeding the next. *Why:* keeps things clean for a task with truly ordered phases (research, then write, then edit). *When:* the phases are always in the same order and don't need to loop back. *Trade-off:* total delay adds up across every stage — no running at the same time — but it's reliable, since each stage has one narrow, clear job.

- **Supervisor** — one router agent hands off work to specialists and collects results. *Why*: handles tasks where the right specialist isn't known ahead of time — the supervisor decides on the fly. *When*: the task type changes per request and needs to be decided dynamically, not a fixed order. *Trade-off*: extra cost/delay for the routing decision itself, plus a new failure — the supervisor can route to the wrong one.
- **Hierarchical** — supervisors of supervisors. *Why*: scales up the supervisor pattern when there are too many specialists for one router to reasonably pick from. *When*: genuinely large systems, with many specialist areas — rarely worth it below that scale. *Trade-off*: stacks the supervisor pattern's cost/delay/routing-error risk, at every level.
- **Peer-to-peer** — agents hand off directly to each other, no central router. *Why*: avoids one router becoming a bottleneck. *When*: agents have clear, non-overlapping handoff conditions they can each recognize on their own. *Trade-off*: harder to reason about and fix than a supervisor setup — there's no single place that shows "the plan," since control is spread out.
- **Parallel** — independent agents run at the same time (using Doc08b's async ideas), results merged together. *Why*: a real speed win when the sub-tasks genuinely don't depend on each other. *When*: the sub-tasks are truly independent — if one needs another's result first, that's sequential, not parallel. *Trade-off*: merging conflicting or inconsistent results from parallel agents is a real design problem, not something automatic.
- **Generator → Critic → Revision** — one agent creates, another checks it, loop until it's good enough (this is Project 4's writer/reviewer pair). *Why*: catches quality problems a single creating agent wouldn't notice about itself. *When*: quality genuinely improves from a separate check, and you can define "good enough" clearly enough for the checker to judge. *Trade-off*: without a hard limit on revision loops, this is Doc07's endless-loop risk again, just with two agents instead of one.

The **Command** tool: today's way of handing work between agents

Older multi-agent LangGraph code often routed between agents using made-up text codes — an agent returns a special word like `"ROUTE_TO_WRITER"`, and a conditional edge then reads it and decides. The **Command** object is today's more direct way: a node can return a **Command** that says both a state update *and* which node to go to next, in one typed object — combining two separate, error-prone pieces (a state update, plus separately-written routing logic) into one clean thing. **Why this is worth learning as the standard, not the older pattern**: it's less fragile (no text to accidentally misspell), and it's what today's LangGraph multi-agent examples and docs are written using.

Shared state design: what goes in the shared state, and what doesn't

Every multi-agent graph has one shared state object that every agent can read, and usually write. The real design question is: what actually belongs in there, versus what should stay private to one agent's own reasoning (its own scratchpad, its own intermediate drafts, its own tool-call history)?

Why putting too much in shared state is a real problem, not just messy: if the Research agent dumps its entire raw scratchpad — every tool call, every half-formed thought — into shared state,

the Writer agent now has to wade through irrelevant noise to find the one paragraph of actual findings it needs, and an unrelated agent might accidentally read (or worse, get confused by) another agent's private reasoning. **Why putting too little in is just as broken:** if agents only share a final one-line result with nothing else, a later agent has no way to ask "why did you conclude that" or catch a mistake early — they can't actually coordinate, just hand off a black box. **How it works in practice:** shared state should hold the things every downstream agent genuinely needs — the task itself, each agent's *finished* output, and routing/status info (whose turn it is, how many revisions have happened) — while each agent's own private working notes stay inside that agent's own node and never get written to the shared object.

```
class SharedState(TypedDict):
    task: str
    research_findings: str      # Research agent's finished output – shared
    draft: str                  # Writer agent's finished output – shared
    revision_count: int         # routing info – shared
    # NOT shared: each agent's raw tool-call scratchpad, kept local to its own node
```

When to use it: this decision has to be made explicitly for every field you add to state — "does every agent need this, or just one?" — rather than defaulting to putting everything in one big shared object because it's easier to wire up.

Error propagation between agents

Doc01 covered error handling for one function: catch a specific error, and either retry, fall back, or raise a clear message up to the caller. In a multi-agent pipeline, that same problem shows up one level bigger — when one agent fails (a tool it called errored out) or succeeds but returns something unusable (an empty draft, a nonsense analysis), what happens to the *whole pipeline*, not just that one function call? **The three real options:** stop the whole pipeline and report the failure clearly (safest, but wastes everything the earlier agents already did); have the supervisor retry just that one failed agent, possibly with a different prompt or a note about what went wrong (works well when the failure looks temporary or fixable); or have the supervisor route *around* the failure — skip that agent, or fall back to a simpler path — when the task can still produce something useful without it. **Why this needs to be a real decision, made on purpose:** the tempting default is to just let a bad result quietly flow downstream — a Writer that gets an empty "research findings" field will still cheerfully write *something*, and now you have a confident-sounding final answer built on nothing, which is much harder to catch than a loud crash. **How it works:** the supervisor (or a dedicated check after each agent) should look at what an agent actually returned, not just assume it worked because no exception was thrown — the same "did this actually succeed" check Doc01 taught for one function, now applied to each handoff in the pipeline.

Cost and latency: multi-agent is not free

Every extra agent in a pipeline is at least one more LLM call — often several, if that agent uses tools or loops internally — which means real added cost (you're paying per token, per call) and real added delay (each call has to finish before the next agent can start, unless you're specifically running things in parallel). A 4-agent pipeline isn't "roughly the same speed" as a single agent; it can easily be 3-4x the cost and wall-clock time of one well-prompted agent doing the whole task in one pass. **When the overhead is worth it:** when the task genuinely has separate phases that would conflict or get confused if forced into one system prompt, when specialist framing measurably improves quality (a dedicated Reviewer catching real mistakes a Writer wouldn't catch in its own output), or when independent pieces can run at the same time and the parallel speed-up outweighs the extra calls. **When it isn't:** this is the same judgment call Doc07 asked you to make one level down — Doc07 asked "does this task need an agent loop at all, or is one tool call enough?"; here the question is "does this task need *multiple* agents, or would one well-organized agent with more tools do the same job for a fraction of the cost and delay?" Both questions have the same trap: reaching for more machinery because it sounds more advanced, not because the task actually needs it. **How to actually know, instead of guessing:** measure it — the Intermediate and Real-world exercises below have you build both versions of the same task and compare real token counts and real wall-clock time, not intuition.

Agent-to-agent communication patterns beyond the `Command` tool

This document focuses on the supervisor pattern — one router agent decides who goes next, using `Command` to hand off work and update state in one step. That's a genuinely common and reliable shape, but it isn't the only one real multi-agent systems use. A **blackboard pattern** has no central router at all: every agent reads from and writes to one shared message log that all agents can see, and each agent decides for itself, by looking at that log, whether it's its turn to act — useful when the right next step genuinely depends on the accumulated state of the whole conversation rather than one router's single decision. A **fixed pipeline** (already named above as the "Sequential" pattern) goes even further the other direction: agent A's output is simply piped straight into agent B as input, in a fixed order, with no routing decision made by anyone, human or agent — simplest to reason about, but only works when the order truly never changes. **Why it's worth knowing these exist, even without building them here:** supervisor-routing is the pattern this document teaches because it's the clearest one to learn from and the most common in production LangGraph code, but "multi-agent system" doesn't automatically mean "supervisor" — recognizing when a blackboard or a fixed pipeline actually fits a task better than adding a supervisor is part of the same judgment this whole document is teaching.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [LangGraph documentation — multi-agent section](#) — search for "multi-agent," "supervisor," and "Command" in the docs.

- [LangChain blog](#) — search for multi-agent architecture posts; read 1-2 recent ones for current patterns.
- [Anthropic — Building Effective Agents](#) — read the orchestrator-worker / evaluator-optimizer sections again now that you can match them directly to the patterns above.

Practice Exercises

Setup for this document's practice code: work inside `11_multi_agent_systems/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install langgraph langchain-openai` .

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py` , with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python env_config_practice.py` .

For this document: - Basic (`paper_design`), Intermediate (`sequential_measure`), and Real-world (`supervisor_compare`) are all the same running comparison — design the single/sequential/supervisor options on paper, then build and measure the sequential version, then build and measure the supervisor version against it — save all three together as `architecture_comparison_practice.py` , with each level as its own section. - Edge cases (`ambiguous_routing`) asks a different question (can the supervisor route correctly when two specialists overlap) — save it as its own topic, `supervisor_routing_practice.py` . - Failure (`convergence_and_cost_cutting`) is its own topic — save it as `convergence_and_cost_cutting_practice.py` .

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — design on paper before touching code

- **What:** design the same task 3 ways (single agent / sequential / supervisor) on paper, guessing which wins on cost/speed/reliability, before writing any code.
- **Why:** this document's central skill is judgment, not syntax — practicing the judgment call *before* code exists is the only way to actually test your intuition against reality afterward.
- **When you'll hit this for real:** the start of every real multi-agent project, including Project 4's own Architecture-Before-Code step.

- **How to practice it:** write one paragraph per design (single/sequential/supervisor) describing the flow, then a one-line prediction for cost, speed, and reliability for each.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — measure the sequential version

- **What:** build the sequential version of a 2-step task (like research → write) and measure real cost/speed against your Basic-level guess.
- **Why:** comparing your prediction to the real number is what actually calibrates your judgment for next time — skipping this step means you never find out if your intuition was right.
- **When you'll hit this for real:** any time you're deciding, for a real project, whether a fixed pipeline is the right shape.
- **How to code it:** two functions chained directly (`write(research(topic))`), with token usage and wall-clock time printed for each stage and the total.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — build and compare the supervisor version

- **What:** build the supervisor version of the same task and compare it against the sequential one on the same test inputs.
- **Why:** this is the direct, apples-to-apples comparison that lets you actually answer "why 4 agents and not 1" with real numbers instead of a guess.
- **When you'll hit this for real:** this document's own Architecture-Before-Code step, which asks for exactly this comparison before Project 4 begins.
- **How to code it:** a small LangGraph with a supervisor node routing to the same two functions as nodes, run on the same 3 test inputs as the sequential version, comparing cost/speed/output quality.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — an ambiguous routing decision

- **What:** a supervisor task where two specialists could both reasonably apply — check whether the supervisor routes correctly, and what happens when it doesn't.
- **Why:** ambiguous routing is where supervisor-pattern systems actually break in production — you need to have watched it happen once, deliberately.
- **When you'll hit this for real:** Project 4's Research vs. Analysis boundary, where a task could plausibly need either first.
- **How to code it:** write a supervisor with two specialists whose descriptions slightly overlap, run 5 ambiguous test prompts through it, and log which one got picked each time.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — a loop that won't converge, and a cost-cutting pass

- **What:** build a generator ↔ critic loop with no guarantee it will ever agree, and watch it fail to stop (carefully, with a real limit in place). Then, separately, try to cut a 4-agent run's cost or speed by 30%+ without breaking correctness.
- **Why:** both are real production concerns — a revision loop that never converges is a direct threat to your uptime, and cost-cutting under a real constraint is a skill interviewers specifically ask about.
- **When you'll hit this for real:** Project 4's Writer/Reviewer pair, and any real system operating under a cost budget.
- **How to code it:** give the critic impossible-to-satisfy criteria, run the loop with a low hard limit, and confirm it exits with a clear "couldn't converge" report. Then profile a working 4-agent run for the most expensive step, and try cutting it (shorter prompts, a cheaper model for one agent, caching) while re-running your test set to confirm correctness held.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Project 4: Multi-Agent System

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: Supervisor → Research Agent → Analysis Agent → Writer Agent → Reviewer Agent, with routing, shared state, and a checker/reviewer step that can send work back for changes.

Requirements:

- A supervisor node that routes to the right specialist(s) based on the task's state, using the `Command` tool for handoffs.
- Shared state that each agent reads and writes, without it filling up with each agent's own unrelated notes (decide what's shared vs. private).
- A reviewer agent that can reject the writer's work and send it back for changes, with a limit on how many revision rounds can happen (no endless generator ↔ critic loop).
- Full LangGraph saved-state, carried over from Project 3.

Inputs: a free-text research/writing task (like "research X and write a short brief").

Outputs: a final, reviewed piece of writing, plus a visible log of which agents ran, in what order, and why the supervisor routed the way it did.

Constraints: no two agents should redundantly call the same tool for the same sub-task; the revision loop must have a limit, and must end with either an accepted result or a clear "couldn't agree" report.

Suggested files:

```
project_4_contentforge_multi_agent/
├─ main.py
├─ graph.py
├─ state.py
├─ agents/
│   ├─ supervisor.py
│   ├─ research_agent.py
│   ├─ analysis_agent.py
│   ├─ writer_agent.py
│   └─ reviewer_agent.py
└─ test_project4.py
```

Functions/Components to build:

- `agents/supervisor.py` → routing logic using `Command`
- one file per specialist agent, each a graph node (or sub-graph)
- `state.py` → the shared state shape, clear about what's global vs. per-agent
- a revision-round counter with a hard limit

Expected Behavior

- A normal task flows: supervisor routes to research → analysis → writer → reviewer; if the reviewer rejects it, back to writer (with a limit); the final accepted result comes back with a full log.
- No two agents redundantly call the same tool for the same sub-task.
- The system reports a clear failure (not an endless loop, not a silent bad answer) if revision never settles within the limit.

Test Cases

Scenario	Expected
A normal task	Full pipeline finishes, reviewer accepts on the first pass
Writer's first draft is deliberately weak (a test case)	Reviewer rejects it, writer improves it, eventually accepted
Revision never settles (a tricky test case)	Limit is hit, a clear "couldn't agree" report, no endless loop
Task is unclear between two specialists	Supervisor routes to one, and you can explain why from its instructions

Break-It / Debug Preview

- The supervisor routes to the wrong agent.
- Two agents redundantly call the same tool.
- Reviewer and writer never agree.
- One agent's private notes leak into shared state and confuse a later agent.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- Full trade-off table for every pattern above · defending "why 4 agents and not 1" · shared vs. private state · redundant-work and state-leaking failures.

☐ You Can Now Build Project 4

Doc11 is everything Project 4 needs on top of Project 3. Go to [project_4_contentforge_multi_agent/](#) and start with its **Setup** section — the step-by-step build guide there (not the Build Task summary above) is what you actually follow.

Move On When

Project 4 finishes a full task through all four agents, survives at least two break scenarios, and you can defend the architecture choice without being asked twice. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"